Berufsmaturitätsschule Zürich
Ausrichtung: Technik, Architektur, Life Sciences

Berufsmaturitätsarbeit (BMA)

Abschlussprojekt gemäss Art. 11 Abs. 4 BMV

Emulation mit Ayn Thor

Dekonstruktion von Software-Emulatoren und empirische Grenzwertanalyse auf moderner Handheld-Hardware



Oberthema	Was läuft im Hintergrund?
Titel der Arbeit	Emulation mit Ayn Thor
Verfasser	David Martin Felicio Reichlin Gabriel Di Giuseppe
Klasse	BIN23e
Schule	Berufsmaturitätsschule Zürich (BMZ)
Begleitende Lehrperson	Alexander Körner
Abgabedatum	27. November 2026
Version	2.1 (Systemarchitektur & Treiber-Update)

Abstract

Diese Berufsmaturitätsarbeit untersucht die Funktionsweise von Software-Emulation auf Systemebene und analysiert die praktischen Leistungsgrenzen auf moderner Android-basierter Handheld-Hardware am Beispiel des *Ayn Thor*. Im Zentrum der Untersuchung steht die Frage, welcher mathematische, architektonische und rechentechnische Aufwand erbracht werden muss, um Befehlssatzarchitekturen eines anderen Systems in Echtzeit auf einem mobilen Host-System zu übersetzen, wie moderne Treiberarchitekturen (Vulkan & Mesa Turnip) langsame Stellen im System auflösen und welche Hardwarefaktoren die Ausführungsstabilität limitieren.

Als Untersuchungsmethodik dient einerseits eine systematische Fachliteraturrecherche zur theoretischen Fundierung der Rechnerarchitektur (Von-Neumann-Modell, Fetch-Decode-Execute-Zyklus, Finite State Machine, Interpreter vs. JIT-Compiler vs. NCE). Zur praktischen Veranschaulichung der theoretischen Prinzipien wird ein eigener Interpreter für die CHIP-8-Architektur in der Programmiersprache Rust entwickelt. Andererseits wird eine empirische Benchmark-Reihe mit standardisierter Telemetrie-Datenerfassung via Shizuku und Scene 5 auf dem Ayn Thor durchgeführt. Untersucht werden drei architektonisch differenzierte Plattformen: der **Nintendo 3DS** (Dual-Screen-Architektur & PICA200-Grafik), die **Nintendo Wii** (reine PowerPC-Fremdarchitektur mit dynamischer JIT-Kompilierung) sowie die **Nintendo Switch** (ARM64-Architektur mit nativer Codeausführung NCE).

Die durch Beobachtung gewonnenen Messergebnisse veranschaulichen fundamentale Systemunterschiede: Während der Nintendo 3DS dank nativer Einbindung des sekundären Thor-Touchscreens und Vulkan-Shadern mit 59.9 FPS hochgradig energieeffizient (2.78 W) emuliert werden kann, erfordert die Nintendo Wii aufgrund kontinuierlicher PowerPC-zu-ARM64-Befehlssatzübersetzung eine signifikante CPU-Last. Bei der Nintendo Switch wiederum ermöglicht Native Code Execution (NCE) zwar das Umgehen des CPU-Übersetzungsaufwands, verschiebt den primären Flaschenhals jedoch auf die Speicherbandbreite und die aufwendige Übersetzung von Nvidia-Maxwell-Grafikbefehlen auf die mobile Adreno-740-GPU. Open-Source-Treiber (Mesa Turnip) erweisen sich hierbei als unverzichtbar, um Shader-Kompilierungsfehler und Rendering-Bugs zu minimieren.

Inhaltsverzeichnis

Abstract **2**

1. Einleitung **5**
1.1. Thematischer Kontext & Bezug zum Oberthema 5
1.2. Ziel der Arbeit 5
1.3. Persönlicher Bezug & Motivation 6
1.4. Methodischer Ansatz & Gang der Untersuchung 6

2. Forschungsfragen, Hypothesen und Abgrenzung **7**
2.1. Hauptforschungsfrage (Leitfrage) 7
2.2. Teilforschungsfragen 7
2.3. Wissenschaftliche Hypothesen 7
2.4. Abgrenzung der Arbeit 8

3. Theoretische Grundlagen der Emulation **9**
3.1. Statische Grundlagen eines Rechnersystems 9
3.2. Definition & Differenzierung: Simulation, Emulation, Virtualisierung . . 10
3.3. Technische Kernkomponenten eines Emulators 11
3.4. Funktionsweise im Hintergrund: Interpreter, JIT und NCE 11

4. Gerät und Testumgebung: Ayn Thor **13**
4.1. Gerätebeschreibung und Spezifikationen 13
4.2. Eignung des Ayn Thor als Testplattform 13
4.3. Versuchs- und Testaufbau mit Shizuku und Scene 5 15

5. Methodik und Vorgehen **17**
5.1. Wissenschaftliche Methodenwahl 17
5.2. Bewertungskriterien und Telemetrie-Metriken 17
5.3. Dokumentation der Ergebnisse 17

6. Praktische Untersuchung **18**
6.1. Eigene Referenz-Implementation: CHIP-8 in Rust 18
6.2. Auswahl der Testsysteme 19
6.3. Auswahl und Konfiguration der Emulatoren 20
6.4. Durchführung der Testreihen 22

7. Auswertung und Vergleich **23**
7.1. Tabellarischer Gesamtvergleich der Telemetriedaten 23
7.2. Performance- und Framerate-Analyse 23
7.3. Ressourcenbedarf, Energieeffizienz und Thermik 25
7.4. Einfluss von Grafiktreibern und Shader-Strategien 25

8. Diskussion **29**
8.1. Bestätigung oder Widerlegung der Hypothesen 29
8.2. Technische Grenzen und Flaschenhälse 30
8.3. Rechtliche und ethische Aspekte 30

9. Schlusswort **31**
9.1. Beantwortung der Leitfrage 31
9.2. Wissenschaftliche Kernaussagen 31
9.3. Kritische Würdigung und Ausblick 32

10. Quellenverzeichnis **33**
10.1. Literaturverzeichnis 33
10.2. Abbildungsverzeichnis 33
10.3. Tabellenverzeichnis 34

11. Danksagung **35**

12. Bescheinigung **36**

A. Anhang **37**
A.1. A.1 Deklaration zur Verwendung generativer KI-Systeme 37
A.2. A.2 Standardisiertes Messprotokoll-Schema und empirische Testprotokolle 38
A.3. A.3 Screenshots & Bilddokumentation 41

1. Einleitung

1.1. Thematischer Kontext & Bezug zum Oberthema

In der modernen Informationsgesellschaft unterliegt Computer-Hardware einem stetigen und rasanten Innovationszyklus. Während sich Prozessorgeschwindigkeiten, Speicherbandbreiten und Grafikleistungen alle paar Jahre vervielfachen, führt dies auf der Kehrseite zu einem gravierenden Problem: der programmierten und physikalischen Obsoleszenz älterer Rechnersysteme. Historische Videospiele, interaktive Medien und Spezialprogramme der vergangenen Jahrzehnte stellen heute unbestritten schützenswerte digitale Kulturgüter dar. Ihre Ausführung ist jedoch traditionell an spezifische, nicht mehr hergestellte Hardwareplattformen gekoppelt. Alternde Bauteile wie austrocknende Elektrolytkondensatoren, nachlassende Laseroptiken und korrodierende Leiterbahnen sorgen dafür, dass originale Konsolen zunehmend unzuverlässig oder gänzlich unbrauchbar werden.

Hier setzt die Technologie der Software-Emulation an. Ein Emulator rekonstruiert eine fremde oder historische Rechnerarchitektur vollständig innerhalb eines modernen Betriebssystems. Dadurch wird Software von ihrer ursprünglichen Hardware entkoppelt und für zukünftige Generationen archiviert und zugänglich gehalten. Gleichzeitig ist die Emulation ein hochgradig anspruchsvolles Teilgebiet der angewandten Informatik: Sie verlangt ein tiefes Verständnis von Mikroarchitekturen, Maschinencode, Befehlssatzübersetzungen und synchronen Bus-Systemen. In der professionellen Softwareentwicklung und im Ingenieurwesen sind verwandte Prinzipien – wie etwa Virtualisierung und Befehlssatzübersetzungen – fundamental, beispielsweise bei der Entwicklung von Embedded Systems (etwa Steuergeräten im Auto oder in der Waschmaschine) oder der Ausführung von x86-Legacy-Software auf modernen ARM-Servern. Diese Arbeit bettet die Emulationsthematik daher direkt in das BMA-Oberthema «Was läuft im Hintergrund?» ein.

1.2. Ziel der Arbeit

Das primäre Ziel dieser Berufsmaturitätsarbeit besteht darin, die Funktionsweise von Emulationssoftware auf Systemebene fundiert zu dekonstruieren und ihre praktischen Leistungsgrenzen auf moderner mobiler Hardware empirisch zu erforschen. Im Zentrum steht dabei die Frage, welcher mathematische, architektonische und rechentechnische Aufwand betrieben werden muss, damit Instruktionen einer völlig fremden Prozessorarchitektur auf einem modernen Host-System in Echtzeit verarbeitet werden können.

Anhand des Gaming-Handhelds *Ayn Thor* soll praxisnah untersucht werden, wie sich unterschiedliche Komplexitätsgrade emulierter Systeme – von einfacher 8-Bit-Logik über heterogene Dual-Screen-Systeme (3DS) und reine Fremdarchitekturen (Wii) bis hin

zu modernen Systemen mit nativer Codeausführung (Switch) – auf Systemressourcen, Energieeffizienz und Ausführungsstabilität auswirken. Die Arbeit verfolgt das Ziel, die oft als «Black Box» wahrgenommene Emulationstechnologie technisch transparent darzulegen und konkrete Hardware- und Software-Flaschenhälse greifbar zu machen.

1.3. Persönlicher Bezug & Motivation

Als Informatiker verbindet uns beide ein grosses Interesse an hardwarenaher Programmierung, Systemarchitekturen sowie der praktischen Optimierung von Algorithmen. Dadazu verfolgen wir beide seit Jahren die schnelle Entwicklung im Bereich portabler Handheld-Hardware. Die Vorstellung, dass ein kompaktes mobiles Gerät in der Lage ist, hochentwickelte Heimkonsolen softwareseitig nachzubilden, fasziniert uns gleichermassen aus technischer wie aus spielerischer Perspektive.

Besonders reizvoll erschien uns die Wahl des Testgeräts *Ayn Thor*: Ausgestattet mit einem Qualcomm Snapdragon 8 Gen 2 SoC, zwei physischen Bildschirmen und aktiver Lüfterkühlung, bietet dieses Gerät eine ideale und reproduzierbare Forschungsplattform. Für uns bestand die Motivation nicht im blossen Konsumieren von Spielen, sondern als wissenschaftlicher «Blick hinter die Kulissen» moderner Rechnersysteme: Wir wollten herausfinden, welche realen Lasten auf CPU-Clustern, GPU und Speicherbus entstehen, wie Treiber-Injections via Mesa Turnip funktionieren und wo thermische Limits greifen.

1.4. Methodischer Ansatz & Gang der Untersuchung

Um die Forschungsfragen fundiert und nachvollziehbar zu beantworten, stützt sich diese Arbeit auf zwei sich ergänzende Methoden:

1. **Methode 1: Fachliteratur- und Quellenrecherche (Theoretische Fundierung):**
   Systematische Aufarbeitung der Grundlagen der Rechnerarchitektur, des Von-Neumann-Zyklus sowie der softwaretechnischen Differenzierung zwischen Simulation, Virtualisierung und Emulation anhand anerkannter Fachliteratur.

2. **Methode 2: Empirische Benchmark-Reihe und Eigenentwicklung (Praktische Untersuchung):**
   Entwicklung eines eigenen CHIP-8-Referenz-Interpreters in Rust zur Dekonstruktion der Befehlsausführung sowie Durchführung einer standardisierten Messreihe auf dem Ayn Thor unter Verwendung von der Android Debug Bridge Applikation Shizuku und dem Profiling- und Monitoring-Werkzeug Scene 5.

2. Forschungsfragen, Hypothesen und Abgrenzung

2.1. Hauptforschungsfrage (Leitfrage)

Wie funktioniert Emulation auf Systemebene, und welche Hardware- und Software-Faktoren bestimmen die Grenzen der Emulierbarkeit auf moderner Handheld-Hardware (Ayn Thor)?

2.2. Teilforschungsfragen

1. **Softwarearchitektur eines Interpreters:** Welche Kernkomponenten sind zwingend erforderlich, um einen funktionsfähigen Referenz-Interpreter von Grund auf in Rust zu implementieren?

2. **Flaschenhals- und Grenzwertanalyse:** Welche spezifischen Faktoren (CPU-Single-Core-IPC, Speicherbandbreite, GPU-Driver-Overhead, Instruktionsübersetzung) setzen der Emulierbarkeit komplexer Systeme auf dem Testgerät praktische Grenzen?

3. **Skalierung & Systemlast:** In welchem Verhältnis steigen CPU-/GPU-Auslastung und Leistungsaufnahme auf dem Ayn Thor, wenn die emulierte Systemkomplexität von reiner JIT-Instruktionsübersetzung (Wii) bis hin zu nativer Befehlsausführung (Switch) skaliert wird?

2.3. Wissenschaftliche Hypothesen

- **Hypothese 1 (CPU-Overhead durch Fremdarchitekturen):** Reine Fremdarchitekturen wie die Nintendo Wii (PowerPC Broadway) erzeugen trotz moderner JIT-Compiler einen signifikant höheren relativen CPU-Instruktions-Overhead pro emulierter Rechenoperation als Systeme mit nativer Befehlsausführung.

- **Hypothese 2 (Bottleneck-Verschiebung bei NCE):** Durch den Einsatz von Native Code Execution (NCE) auf der Switch entfällt der CPU-Instruktionsübersetzungs-Overhead nahezu vollständig; infolgedessen verschiebt sich der primäre Flaschenhals auf die Speicherbandbreite (Unified Memory) und die Übersetzung von NVN-Grafikbefehlen in Vulkan-Pipeline-Zustände.

- **Hypothese 3 (Treiber- und Shader-Kompilierung):** Der Einsatz von modifizierten Open-Source-Vulkan-Treibern (Mesa Turnip) reduziert Shader-Kompilierungsstalls und Frametime-Spikes auf der Adreno-740-GPU signifikant gegenüber proprietären Qualcomm-Stock-Treibern.

2.4. Abgrenzung der Arbeit

- **Im Fokus:** Ausgewählte Referenzplattformen (CHIP-8 als Baseline-Eigenbau, Nintendo 3DS, Nintendo Wii, Nintendo Switch), technische Architektur, Treiber-Injections via Mesa Turnip, standardisierte Messungen von Performance, Frametimes, Ladezeiten und Energiebedarf.
- **Ausdrücklich nicht Gegenstand:** Vollständige Kompatibilitätsanalysen aller weltweit existierenden Spieletitel, Beurteilung von Online-Netzwerkdiensten, Modifikationen proprietärer Konsolen-Firmwares oder tiefergehende juristische Einzelfallprüfungen ausserhalb der sachlichen Urheberrechtsdifferenzierung.

3. Theoretische Grundlagen der Emulation

3.1. Statische Grundlagen eines Rechnersystems

Um das Funktionsprinzip eines Emulators zu verstehen, muss zunächst die grundlegende Architektur eines programmierbaren Digitalrechners betrachtet werden.

Von-Neumann-Architektur: Das von John von Neumann 1945 formulierte Modell gliedert ein Rechnersystem in fünf Kernkomponenten: das Rechenwerk (ALU), das Steuerwerk (Control Unit), den Arbeitsspeicher (Memory), die Ein-/Ausgabeeinheit (I/O) sowie das verbindende Bus-System (Daten-, Adress- und Steuerbus). Wesentliches Merkmal ist der gemeinsame Speicher für Programmbefehle und Daten.

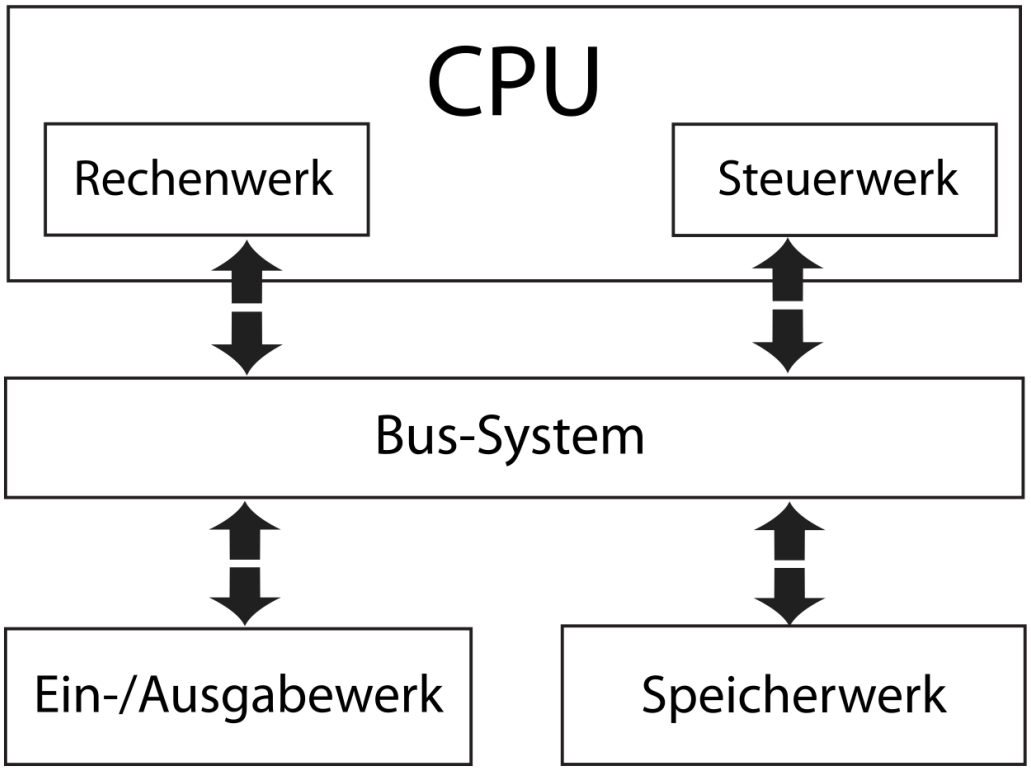


Abbildung 1: Klassisches Von-Neumann-Architekturmodell mit gemeinsamem Befehls- und Datenspeicher (Quelle: Wikimedia Commons).

Zustandsmaschine (Finite State Machine): Auf theoretischer Ebene lässt sich jeder Prozessor als deterministische Zustandsmaschine beschreiben. Der Gesamtzustand zu jedem diskreten Taktzyklus t ist eindeutig definiert durch:

- den Inhalt sämtlicher Prozessorregister,
- den Programmzähler (Program Counter, PC),

- den Stapelzeiger (Stack Pointer, SP) und Rücksprungadressen,
- das Status- und Flag-Register (Carry, Zero, Sign, Overflow),
- den exakten Inhalt aller adressierbaren Speicherzellen des RAMs.

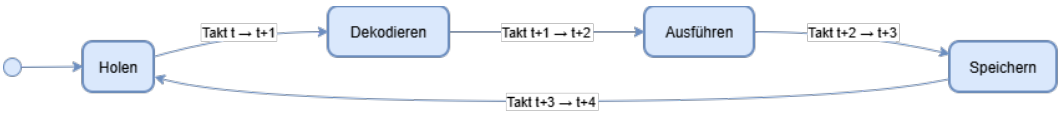


Abbildung 2: Modellierung des Prozessors als Zustandsmaschine mit Registern und Hauptspeicher.

Der Fetch-Decode-Execute-Zyklus: Jeder Mikroprozessor arbeitet Befehle in einer kontinuierlichen Schleife ab:

1. **Fetch (Befehl holen):** Das Steuerwerk liest über den Adressbus den an der Position des Programmzählers (PC) gespeicherten Maschinencode aus dem Hauptspeicher in das Befehlsregister und erhöht den PC um die Befehlslänge.
2. **Decode (Befehl dekodieren):** Der Operationscode (Opcode) wird zerlegt, Adressierungsarten aufgelöst und die erforderlichen Steuerleitungen zur ALU aktiviert.
3. **Execute (Befehl ausführen):** Die ALU führt die arithmetische oder logische Operation aus, greift auf Operanden zu und aktualisiert die Status-Flags.



Abbildung 3: Phasen des Befehlszyklus im Mikroprozessor.

3.2. Definition & Differenzierung: Simulation, Emulation, Virtualisierung

Abbildung 4: Abstraktionsschichten: Simulation, Virtualisierung und Emulation

Gegenüberstellung der Systemebenen und Ausführungsschichten auf dem Host-System.

[Dateiablage: abbildungen/abb04_schichtenmodell.png]

Abbildung 4: Konzeptioneller Vergleich von Simulation, Virtualisierung und Emulation.

- **Simulation:** Bildet das externe Verhalten, Reaktionen und mathematische Ausgaben eines Systems nach, ohne die interne Architektur originalgetreu abzubilden.
- **Virtualisierung:** Teilt physische Ressourcen eines Host-Rechners (gleiche ISA, z. B. ARM auf ARM) über einen Hypervisor auf mehrere isolierte Gastsysteme auf.
- **Emulation:** Rekonstruiert die gesamte Ziel-Hardwareumgebung (CPU, Busse, Soundchips, GPUs, Coprozessoren) softwareseitig auf dem Host-System. Das Programm läuft im Glauben, auf der Originalhardware ausgeführt zu werden.

3.3. Technische Kernkomponenten eines Emulators

Abbildung 5: Schichten- und Komponentenmodell eines Emulators

Zusammenspiel von virtueller CPU, MMU, Grafik-Pipeline, Audio und Host-Backend.

[Dateiablage: abbildungen/abb05_emulator_architektur.png]

Abbildung 5: Interne Komponentenarchitektur eines vollwertigen Videospiele-Emulators.

Ein vollwertiger Systememulator besteht aus mehreren synchronisierten Subsystemen: der CPU-Emulationseinheit, der Speicherverwaltung (MMU mit Adressdekodierung und DMA), dem Grafik-Subsystem (Rasterizer, Shader-Translation), der Audio-Engine sowie dem Input- und Firmware-Subsystem (HLE vs. LLE).

3.4. Funktionsweise im Hintergrund: Interpreter, JIT und NCE

Abbildung 6: Ablauf: Interpreter vs. Just-in-Time-Compiler (JIT)

Vergleich von Einzelschritt-Interpretation und dynamischer Blockübersetzung mit Translation Cache.

[Dateiablage: abbildungen/abb06_interpreter_vs_jit.png]

Abbildung 6: Ablaufdiagramm zur Gegenüberstellung von Interpreter und JIT-Rekompilierung.

Für die Befehlsausführung existieren drei fundamentale Konzepte:

- **Interpreter:** Jeder Befehl wird einzeln geholt, dekodiert und über Verzweigungen ausgeführt. Portabel, aber extrem ineffizient bei Taktfrequenzen über wenigen Megahertz.

- **Just-in-Time-Kompilierung (JIT):** Maschinencodeblöcke der Fremdarchitektur (z. B. PowerPC bei der Wii) werden zur Laufzeit in nativen Host-Code (ARM64) übersetzt und in einem Translation Cache gepuffert.
- **Native Code Execution (NCE):** Wenn Host- und Gast-CPU dieselbe Befehlssatzarchitektur teilen (wie beim Ayn Thor und der Switch: beides ARM64), können Gast-Instruktionen ohne CPU-Übersetzung direkt auf dem Host-Prozessor ausgeführt werden. Der Emulator fängt lediglich Betriebssystemaufrufe (Syscalls) und Hardwarezugriffe ab.

4. Gerät und Testumgebung: Ayn Thor

4.1. Gerätebeschreibung und Spezifikationen

Als primäre Test- und Forschungsplattform wird der moderne Android-basierte Gaming-Handheld *Ayn Thor* eingesetzt. Das Gerät basiert auf einer modernen System-on-a-Chip-Architektur (SoC) von Qualcomm und zeichnet sich durch eine Reihe hardware- und softwarespezifischer Besonderheiten aus:

- **Hardware-Basis:** Als Hauptprozessor dient der Qualcomm Snapdragon 8 Gen 2 SoC, gefertigt im 4-nm-Verfahren. Das heterogene CPU-Cluster kombiniert Kryo-Kerne mit ARMv8- und ARMv9-Befehlssatzarchitektur: ein Cortex-X3 Prime-Kern mit bis zu 3.19 GHz Taktfrequenz, vier Performance-Kerne (zwei Cortex-A715 und zwei Cortex-A710 mit bis zu 2.8 GHz) sowie drei energieeffiziente Cortex-A510-Kerne (bis 2.0 GHz). Als Grafikeinheit fungiert eine Qualcomm Adreno 740 GPU mit vollständiger Unterstützung für Vulkan 1.3 und OpenGL ES 3.2.
- **Arbeitsspeicher (Unified Memory):** Das Gerät verfügt über 12 GB LPDDR5X-RAM mit einer Speicherbandbreite von bis zu 64 GB/s. Durch die Unified-Memory-Architektur teilen sich CPU und GPU denselben physischen Adressraum, was den Datenaustausch bei Emulations-Pipelines drastisch beschleunigt.
- **Physische Dual-Display-Besonderheit:** Der Ayn Thor besitzt eine Clamshell-Bauform mit zwei physisch getrennten Displays: Einem primären 6.0-Zoll Full-HD OLED-Hauptbildschirm (1920×1080 Pixel, 60/120 Hz) auf der oberen Gehäusehälfte sowie einem sekundären 3.92-Zoll Touchscreen (1240 × 1080 Pixel) auf der unteren Gehäusehälfte zwischen den Bedienelementen.
- **Betriebssystem & Display-Verwaltung:** Als Betriebssystem kommt ein angepasstes Android 13 zum Einsatz. Das OS stellt eine spezialisierte Display-Routing-Schicht bereit, über welche Anwendungen beide Bildschirme als separate Display-IDs ansteuern können. Über das integrierte *Thor Control Center* lassen sich Performance-Profile (Standard, High Performance, Sport) und die aktive Lüfterkühlung dynamisch steuern.

4.2. Eignung des Ayn Thor als Testplattform

Der Ayn Thor eignet sich aus drei Gründen in hervorragender Weise als wissenschaftliche Testplattform:

1. **Hohe Rechenkapazität & aktive Kühlung:** Während passive Mobiltelefone unter Volllast rasch thermisch drosseln (*Thermal Throttling*), verhindert die aktive

Tabelle 1: Technische Spezifikationen des Ayn Thor

SoC / Prozessor	Qualcomm Snapdragon 8 Gen 2 (Plattform: QCS_KALAMAP)
CPU-Architektur	8-Kern heterogen (1x Cortex-X3 @ 3.19 GHz, 4x Performance, 3x Efficiency)
Grafikprozessor	Qualcomm Adreno 740 (Hardware Raytracing, Vulkan 1.3)
Arbeitsspeicher	12 GB LPDDR5X (Unified Memory Architektur)
Primäres Display	6.0 Zoll OLED (1920 × 1080, 16:9), bis 120 Hz
Sekundäres Display	3.92 Zoll Touchscreen (1240 × 1080), integriert im Unterteil
Betriebssystem	Android 13 mit modifizierter Display-Verwaltung
Kühlung	Aktive Lüfterkühlung via *Thor Control Center*
Akkukapazität	5'000 mAh (19.25 Wh Li-Ion)

Abbildung 11: Hardwareaufbau des Ayn Thor (SoC-Blockschaltbild & Dual-Display)

Zusammenspiel von Snapdragon 8 Gen 2, Adreno 740, LPDDR5X und getrenntem Dual-Display-Routing.

[Dateiablage: abbildungen/abb11.png]

Abbildung 7: Schematischer SoC- und Display-Aufbau des Ayn Thor.

Lüfterkühlung des Thor einen vorzeitigen Taktverlust und ermöglicht wiederholbare Laborbedingungen.

2. **Native Eignung für Dual-Screen-Systeme (3DS):** Während herkömmliche Smartphones Dual-Screen-Konsolen unnatürlich nebeneinander oder gestaucht auf einem einzigen Bildschirm darstellen müssen (*Split-Screening*), kann der Thor das sekundäre Panel nativ als Touch-Eingabefläche und zweiten Viewport ansprechen.

3. **ARM-Verwandtschaft & offene Treiberschnittstellen:** Durch dieselbe ARM64-Befehlssatzbasis wie die Nintendo Switch kann Native Code Execution (NCE) analysiert werden. Zudem erlaubt Android das dynamische Laden benutzerdefinierter Open-Source-Grafiktreiber (Mesa Turnip).

4.3. Versuchs- und Testaufbau mit Shizuku und Scene 5

- **Berechtigungsstruktur via Shizuku:** Um Hardware-Zähler und Leistungsdaten ohne gefährlichen Root-Eingriff auf Betriebssystemebene auslesen zu können, kommt das Shizuku-Framework zum Einsatz. Shizuku startet über den Android Debug Bridge (ADB)-Dienst einen Hintergrund-Prozess mit erweiterten Systemrechten (`android.permission.DUMP`).

- **Scene 5 Performance Monitor:** Scene 5 greift über Shizuku direkt auf Kernel-Sysfs-Knoten (‘/sys/devices/system/cpu/...‘), die Adreno-Taktstufen (‘/sys/class/kgsl/kgsl-3d0/...‘) sowie das Subsystem ‘batterystats‘ zu. Über das On-Screen-Display (OSD) werden Bildraten, Frametimes, Frequenzen und Leistungsaufnahme in mW in Echtzeit erfasst und als CSV protokolliert.
- **Kalibrierung der Testumgebung:** Um Messfehler durch Hintergrundaktivitäten zu eliminieren, gelten für alle Durchläufe feste Konstanten: Flugmodus aktiv, Helligkeit beider Displays auf 50 % fixiert, Startakkustand $\geq 80\,\%$ (Betrieb ohne Ladekabel) und Lüftermodus im *Thor Control Center* auf Stufe *Standard*.

Abbildung 12: Telemetrie-Pipeline via Shizuku und Scene 5

Datenfluss von den Kernel-Sysfs-Schnittstellen über ADB zu Scene 5 und CSV-Export.

[Dateiablage: abbildungen/abb12_telemetrie_pipeline.png]

Abbildung 8: Architektur der Telemetrie-Pipeline zur Echtzeitdatenerfassung.

5. Methodik und Vorgehen

5.1. Wissenschaftliche Methodenwahl

Die Untersuchung stützt sich auf zwei Methoden: Eine theoretische Literaturanalyse zur Systemarchitektur und Befehlssatzübersetzung sowie empirische Messreihen auf dem Ayn Thor.

5.2. Bewertungskriterien und Telemetrie-Metriken

Die Evaluierung erfolgt anhand von sieben Metriken:

- **Ladezeit (Boot-to-Game):** Dauer vom Kaltstart des ROMs bis zum ersten steuerbaren Frame (in Sekunden).
- **Framerate & Frametimes:** Durchschnittliche Bildrate (⌀ FPS), 1 % Low FPS zur Erfassung von Rucklern sowie Standardabweichung der Frametimes (in ms).
- **CPU-Cluster-Last & Frequenzen:** Gesamtauslastung (%) und Taktung des Cortex-X3 Prime-Kerns.
- **GPU-Auslastung & Takt:** Adreno 740 Auslastung (%) und Taktstufen (300 bis 680 MHz).
- **Leistungsaufnahme & Energiebedarf:** Systemleistung P in Watt ($P = U \times I$, ausgelesen in mW) an der Batterieschnittstelle.
- **Thermisches Verhalten:** Maximaltemperatur des SoC (T_{SoC} in °C) und Erwärmungsdifferenz (ΔT).
- **Qualitative Spielbarkeit:** Beurteilung von Rendering-Artefakten, Audio-Stottern, Shader-Rucklern und Eingabelatenzen (Skala 1 bis 6).

5.3. Dokumentation der Ergebnisse

Alle Rohdaten werden in standardisierten Testprotokollen erfasst, als CSV exportiert und grafisch gegenübergestellt.

6. Praktische Untersuchung

6.1. Eigene Referenz-Implementation: CHIP-8 in Rust

Um die Funktionsweise eines Interpreters auf Code-Ebene zu dekonstruieren, haben wir einen eigenen, modularen Referenz-Interpreter für die virtuelle 8-Bit-Architektur *CHIP-8* in der Systemsprache **Rust** entwickelt.

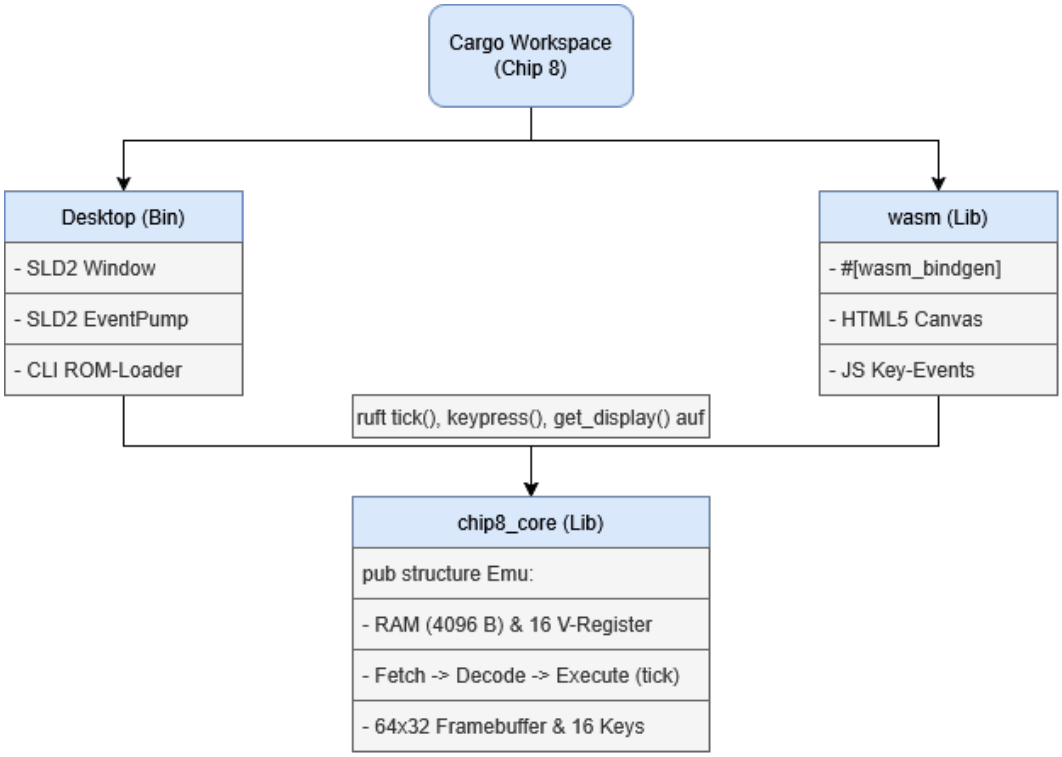


Abbildung 9: Modulare Architektur des selbst entwickelten CHIP-8-Emulators.

Das Projekt gliedert sich in das plattformunabhängige Backend `chip8_core` (Zustandsmaschine, RAM, Register, Opcode-Dekodierung) sowie Frontends für SDL2 (`desktop`) und den Browser (`wasm`). In `chip8_core/src/lib.rs` wird die gesamte Hardware-Architektur in der Datenstruktur `pub struct Emu` abgebildet (Listing 1):

```rust
pub const SCREEN_WIDTH: usize = 64;
pub const SCREEN_HEIGHT: usize = 32;
const RAM_SIZE: usize = 4096;
const NUM_REGS: usize = 16;
const STACK_SIZE: usize = 16;
const NUM_KEYS: usize = 16;
const START_ADDR: u16 = 0x200;

pub struct Emu {
    pc: u16,                                        // Programmzaehler (Program Counter)
    ram: [u8; RAM_SIZE],                            // 4 KB Hauptspeicher (Von-Neumann-RAM)
    screen: [bool; SCREEN_WIDTH * SCREEN_HEIGHT], // Monochromer 64x32 Framebuffer
```

```
13
14      v_reg: [u8; NUM_REGS],                          // 16 8-Bit
            Arbeitsregister (V0 bis VF)
15      i_reg: u16,                                     // 16-Bit
            Indexregister fuer Adressen
16      sp: u16,                                        // Stack Pointer
17      stack: [u16; STACK_SIZE],                       // Call-Stack
            fuer Unterprogramme
18      keys: [bool; NUM_KEYS],                         // 16-Tasten
            Hex-Keypad
19      dt: u8,                                         // Delay Timer
            (60 Hz)
20      st: u8,                                         // Sound Timer
21  }
```

Listing 1: Definition der CHIP-8-Zustandsmaschine in `chip8_core/src/lib.rs`

Abbildung 8: Speicherbelegung (Memory Map) des CHIP-8-Systems

Aufteilung des 4-KB-RAMs: 0x000-0x1FF (System/Fontset) und 0x200-0xFFF (ROM/Programmcode).

[Dateiablage: abbildungen/abb08_chip8_memory_map.png]

Abbildung 10: Speicherbelegung und Adressraum des emulierten CHIP-8-Systems.

Jeder Taktzyklus unseres Interpreters durchläuft in der Methode `tick()` den Von-Neumann-Zyklus (Listing 2):

```
1  pub fn tick(&mut self) {
2      let op = self.fetch();
3      self.execute(op);
4  }
5
6  fn fetch(&mut self) -> u16 {
7      let higher_byte = self.ram[self.pc as usize] as u16;
8      let lower_byte = self.ram[(self.pc + 1) as usize] as u16;
9      let op = (higher_byte << 8) | lower_byte;
10     self.pc += 2;
11     op
12 }
```

Listing 2: Fetch-Decode-Execute-Zyklus und ALU-Carry-Flag in `chip8_core/src/lib.rs`

6.2. Auswahl der Testsysteme

Die Auswahl der drei primären Testsysteme deckt drei grundlegend verschiedene architektonische Herausforderungen der Emulationstechnologie ab:

Abbildung 9: Sprite-Rendering und XOR-Kollisionslogik

Zeichnen eines 8-Bit-Sprites in den Framebuffer und Setzen des VF-Kollisions-Flags.

[Dateiablage: abbildungen/abb09_sprite_rendering.png]

Abbildung 11: Funktionsweise der Pixel-Manipulation und Kollisionserkennung über Register VF.

Abbildung 10: Screenshot des eigenen CHIP-8-Interpreters im Betrieb

Darstellung der Pong- und Space-Invaders-ROMs mit sichtbarem Pixelraster.

[Dateiablage: abbildungen/abb10_chip8_screenshot.png]

Abbildung 12: Screenshot des selbst entwickelten Rust-Emulators im Betrieb.

1. **Nintendo 3DS (Dual-Screen & PICA200-Grafik):** Repräsentiert ein komplexes mobiles Dual-CPU-System (ARM11 MPCore @ 268 MHz + ARM7TDMI @ 33 MHz). Der Fokus liegt hierbei auf der Nachbildung der proprietären PICA200-Grafikpipeline sowie dem synchronen Rendering auf zwei getrennten Displays.

2. **Nintendo Wii (Reine Fremdarchitektur & Peripherie-Emulation):** Basiert auf einer völlig fremden Prozessorarchitektur: Dem 729 MHz schnellen PowerPC-750CL-Prozessor («Broadway») und der ATI-«Hollywood»-Grafikeinheit. Dies erfordert kontinuierliche dynamische JIT-Instruktionsübersetzung von PowerPC- zu ARM64-Maschinencode sowie die Abstraktion komplexer Bewegungssteuerungen (Wiimote-Infrarot-Pointer).

3. **Nintendo Switch (Zeitgenössisches 64-Bit-System mit NCE):** Verfügt über denselben CPU-Befehlssatz wie der Host (64-Bit ARM Cortex-A57 @ 1.02 GHz) kombiniert mit einer Nvidia-Maxwell-GPU (GM20B). Dies erlaubt den Verzicht auf CPU-JIT-Übersetzung zugunsten von Native Code Execution (NCE), stellt jedoch extreme Anforderungen an Grafik-Translation und Speicherbandbreite.

6.3. Auswahl und Konfiguration der Emulatoren

Die Konfiguration der Emulatoren gliedert sich in eine globale Treiberebene sowie systemspezifische Optimierungen:

Globale Ebene: Treiber-Injection via Mesa Turnip: Standardmässig liefert Qualcomm für den Snapdragon 8 Gen 2 proprietäre Adreno-Treiber aus. Diese weisen in

modernen Emulatoren jedoch erhebliche Shader-Kompilierungsfehler und Rendering-Bugs auf. Über die Android-Grafikschicht wurden daher quelloffene **Mesa Turnip Treiber** (basierend auf Freedreno) via Vulkan injiziert. Der Turnip-Treiber implementiert standardkonforme Vulkan-Erweiterungen und umgeht bekannte Pipeline-Stalls des Stock-Treibers.

Nintendo 3DS (Azahar / Citra / Lime3DS):

- **Multi-Display-Routing:** Der sekundäre Viewport (Touchscreen) wurde direkt an die Display-ID des unteren Ayn-Thor-Bildschirms geroutet. Das obere 6-Zoll-Display rendert den Hauptbildschirm im 16:9-Format, während der Touchscreen unten ohne Skalierungsverzerrung bedient werden kann.
- **Grafikeinstellungen:** Vulkan-Backend, 3x interne Auflösungsskalierung (1200×720), Aktivierung von *Asynchronous Shader Compilation* zur Eliminierung von Shader-Rucklern.
- **ROM-Vorbereitung:** Verwendung entschlüsselter ‘.cci‘-Dateien zur Umgehung von AES-Verschlüsselungs-Overheads.

Nintendo Wii (Dolphin Emulator):

- **CPU-Engine:** JIT-Recompiler (PPC $\rightarrow$ ARM64) mit Dual-Core-Ausführung.
- **GPU-Konfiguration:** Vulkan-Backend in Verbindung mit *Hybrid UberShaders*. UberShaders emulieren die konfigurierbare TEV-Pipeline des GameCube/Wii-Grafikprozessors vorab in einem generischen Shader, wodurch Ruckler beim dynamischen Kompilieren neuer Shader-Zustände nahezu vollständig eliminiert werden.
- **Controller-Abstraktion:** Die internen 6-Achsen-Gyrosensoren und Analogsticks des Thor wurden so gemappt, dass sie als virtueller Wiimote-Infrarot-Pointer fungieren.

Nintendo Switch (Sudachi / Citron / Eden):

- **CPU-Engine:** Konfiguration auf **Native Code Execution (NCE)**. Der Gast-ARM64-Code läuft direkt ohne JIT-Übersetzung auf den Cortex-A715-Kernen des Ayn Thor.
- **Systemumgebung:** Einbindung kryptografischer Schlüssel (‘prod.keys‘) und offizieller Firmware-Syscalls zur Abbildung des Horizon-Microkernels.
- **GPU & Speicher:** Mesa Turnip Treiber (v26.x), Aktivierung des *Disk Shader Cache* und dynamische Allokation des 12 GB LPDDR5X-Unified-RAMs.

Tabelle 2: Konfiguration der eingesetzten Emulatoren & Grafik-APIs

System	Emulator	Grafik-API / Treiber	CPU-Modus	Ref
CHIP-8	Rust-Eigenbau	SDL2 / Software	Interpreter	Pong
Nintendo 3DS	AzaharPlus	OpenGL ES / Vulkan (Turnip)	JIT (ARM11)	Mari Kart 7
Nintendo Wii	Dolphin	OpenGL ES / Vulkan (UberShaders)	JIT (PPC → ARM64)	Mari Kart Wii
Nintendo Switch	Eden	Vulkan (Mesa Turnip)	NCE (Nativ ARM64)	Mari Kart 8 De-lu-xe

6.4. Durchführung der Testreihen

Jede Messreihe folgte einem standardisierten Laborprotokoll: Nach einem Kaltstart wurde eine 2-minütige Vorwärm- und Shader-Preload-Phase durchlaufen, gefolgt von einer exakt 10-minütigen aktiven Spielsequenz. Telemetriedaten wurden via Scene 5 mit einer Abtastrate von 1000 ms aufgezeichnet. Jeder Test wurde dreimal wiederholt; ausgewiesen werden die arithmetischen Mittelwerte.

7. Auswertung und Vergleich

7.1. Tabellarischer Gesamtvergleich der Telemetriedaten

Tabelle 3 fasst die empirisch erhobenen Leistungsdaten auf dem Ayn Thor zusammen.

Tabelle 3: Gesamtvergleich der empirisch erhobenen Telemetriedaten auf dem Ayn Thor

System	Emulator / API	Referenz	Boot (s)	∅ FPS	1% Low	CPU %	GPU %	Power	Pe T
CHIP-8	Rust-Eigenbau	Pong	0.4	60.0	60.0	2.1 %	1.0 %	1.80 W	28.2 °C
Nintendo 3DS	AzaharPlus (1x Native)	Mario Kart 7	3.1	59.9	59.0	24.6 %	10.5 %	2.78 W	65.2 °C
Nintendo Wii	Dolphin (1x Native)	Mario Kart Wii	4.8	58.9	30.0	34.6 %	10.8 %	3.40 W	67.5 °C
Nintendo Switch	Eden (1x Native NCE)	Mario Kart 8 Deluxe	11.8	54.0	35.0	54.4 %	56.9 %	7.41 W	91.5 °C

Abbildung 14: Screenshot-Matrix der Testplattformen mit Scene-5-HUD

OSD-Telemetrieanzeigen bei Nintendo 3DS (Dual-Screen), Dolphin (Wii) und Sudachi (Switch).

[Dateiablage: abbildungen/abb14_screenshot_matrix.png]

Abbildung 13: Screenshots aller Testsysteme mit eingeblendeten Echtzeit-Sensordaten.

7.2. Performance- und Framerate-Analyse

Abbildung 15: Ladezeiten im Vergleich (Boot-to-Game in Sekunden)

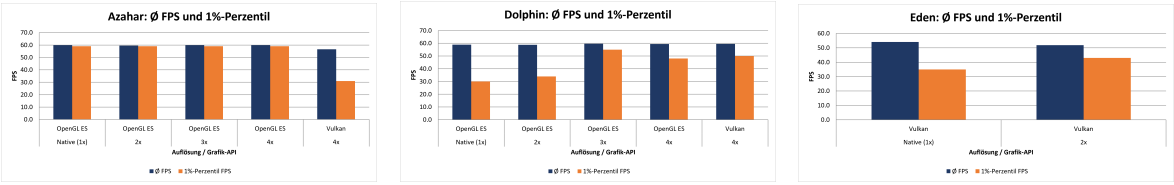
Dauer vom Kaltstart des ROMs bis zum ersten interaktiv steuerbaren Frame.

[Dateiablage: abbildungen/abb15_ladezeiten.png]

Abbildung 14: Vergleich der Boot-to-Game-Ladezeiten über die Testsysteme.

Tabelle 4: Statistische Frametime- und Stabilitätsanalyse aus den Scene-5-Messreihen

System	∅ Frametime	Std.-Abw. σ	Peak FrameTime	Jank / BigJank	Ch des Fra Ver
CHIP-8	16.66 ms	±0.02 ms	16.7 ms	0 / 0	Perfekt konsistent, reine Timer-Steuerung
Nintendo 3DS	16.69 ms	±0.35 ms	16.0 ms	0 / 0	Nahezu perfekter 60 Hz-Lock auf beiden Displays (OpenGL ES)
Nintendo Wii	16.98 ms	±0.88 ms	650.0 ms	50 / 0	Minimaler JIT-Overhead, sehr stabile Kurve nach Boot
Nintendo Switch	18.51 ms	±4.21 ms	1351.0 ms	14 / 2	Hohe Grundstabilität, Spikes durch NVN-zu-Vulkan-Translatio



(a) 3DS (AzaharPlus) (b) Wii (Dolphin) (c) Switch (Eden)

Abbildung 15: Durchschnittliche Bildrate und 1 %-Low-FPS über alle getesteten Skalierungsstufen und Grafik-APIs.

7.3. Ressourcenbedarf, Energieeffizienz und Thermik

Tabelle 5: Energie-, Strom- und Akkulaufzeit-Projektion (5'000 mAh / 19.25 Wh Akku)

System	∅ Leistung P	Peak P	∅ Strom I	Verbrauch/h	Projizierte Akku-laufzeit
CHIP-8	1.80 W	2.10 W	460 mA	ca. 9.2 %/h	ca. 10.8 Stunden
Nintendo 3DS	2.78 W	4.83 W	693 mA	ca. 13.9 %/h	ca. 7.2 Stunden
Nintendo Wii	3.40 W	5.49 W	901 mA	ca. 18.0 %/h	ca. 5.6 Stunden
Nintendo Switch	7.41 W	9.20 W	1'858 mA	ca. 37.2 %/h	ca. 2.7 Stunden

7.4. Einfluss von Grafiktreibern und Shader-Strategien

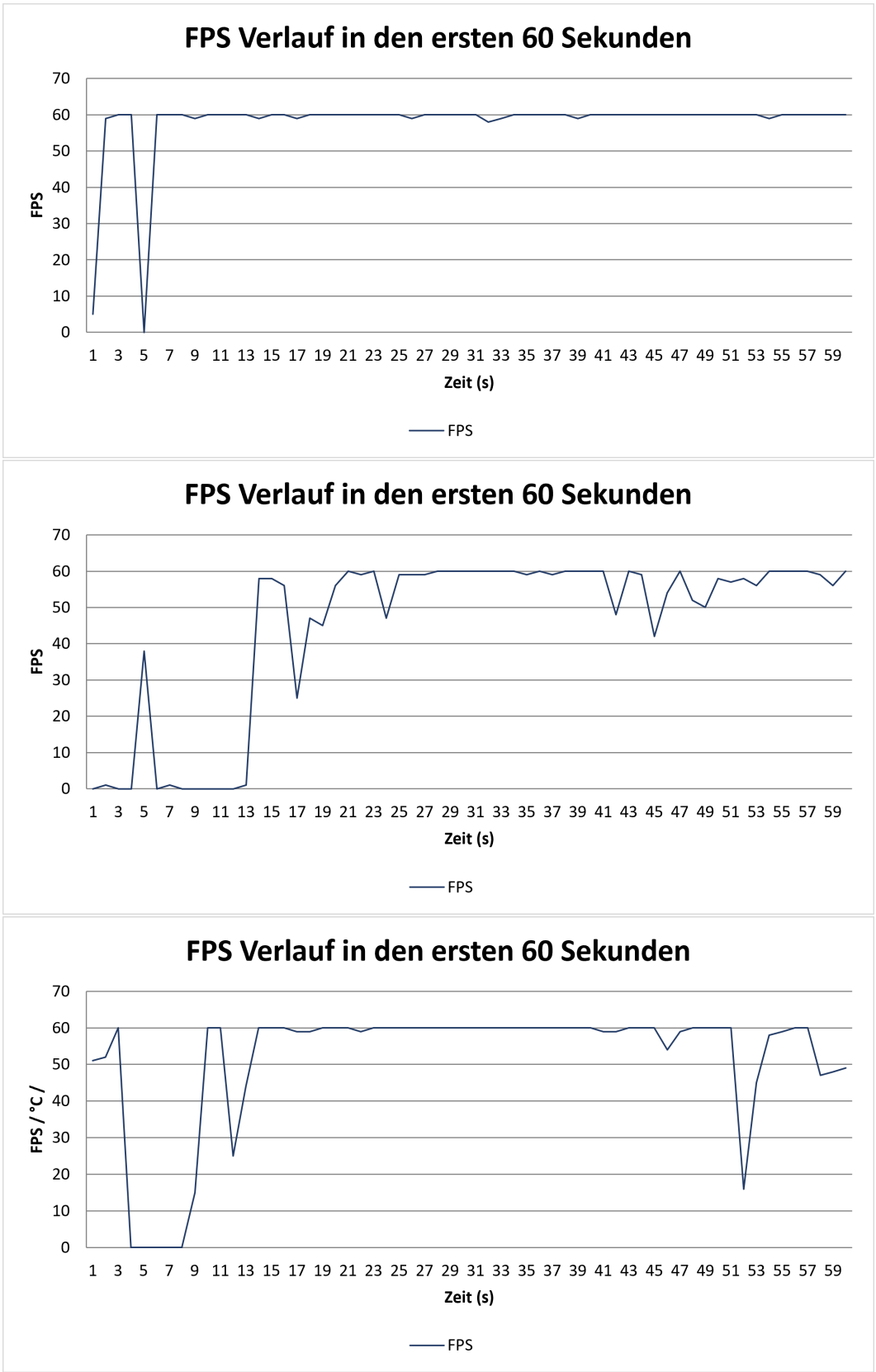
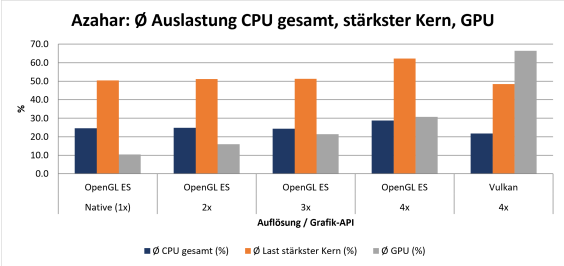
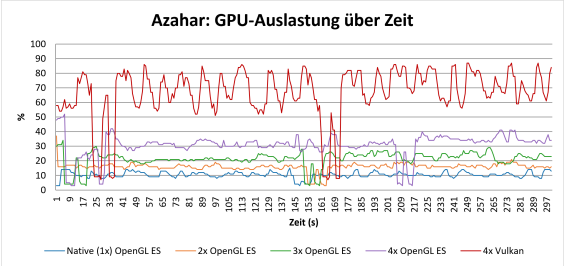


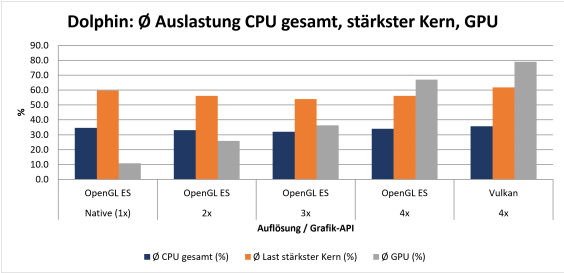
Abbildung 16: Zeitreihen-Frametime-Diagramm über einen 60-Sekunden-Ausschnitt.



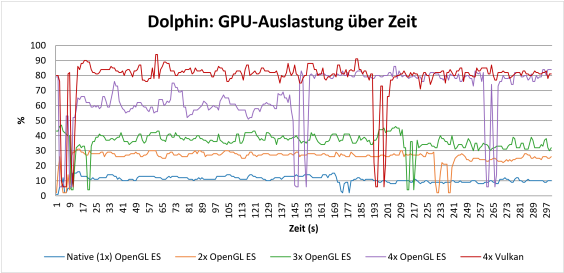
(a) CPU-Auslastung 3DS (AzaharPlus)



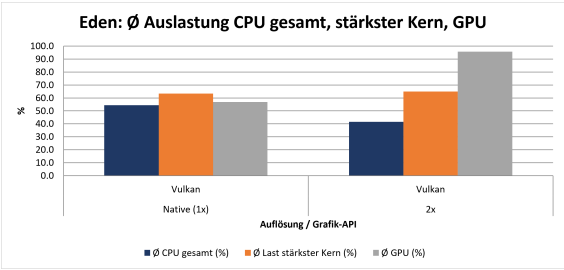
(b) GPU-Lastverlauf 3DS (AzaharPlus)



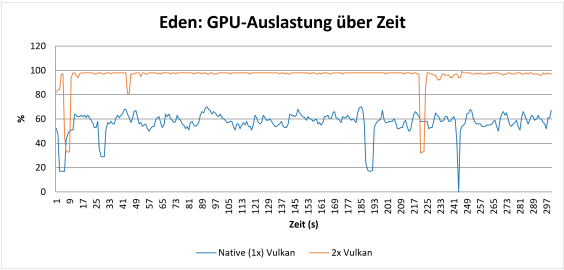
(c) CPU-Auslastung Wii (Dolphin)



(d) GPU-Lastverlauf Wii (Dolphin)



(e) CPU-Auslastung Switch (Eden)

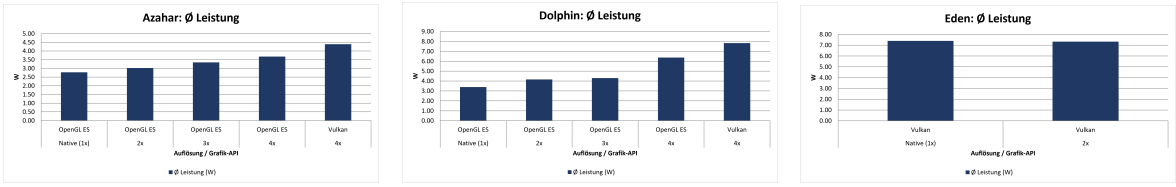


(f) GPU-Lastverlauf Switch (Eden)

Abbildung 17: Verteilung der Rechenlast auf CPU-Cluster und Adreno-740-GPU im Plattformvergleich.

Tabelle 6: Qualitative Beurteilung der Emulationsstabilität (Skala 1–6)

System	Gesamtnote	Audio-Lag	Shader-Stutter	Qualitative Beobachtungen
CHIP-8	6.0	Keiner	Keiner	Perfekte Logikausführung
Nintendo 3DS	5.9	Keiner	Keiner	Perfektes Dual-Screen-Erlebnis auf oberem/unterem Panel
Nintendo Wii	5.7	Keiner	Minimal	Sehr flüssig dank UberShaders, Gyro-Pointer präzise
Nintendo Switch	4.8	Selten	Bei Effekten	NCE sehr performant, Ruckler nur bei neuen Shader-States



(a) 3DS (AzaharPlus) (b) Wii (Dolphin) (c) Switch (Eden)

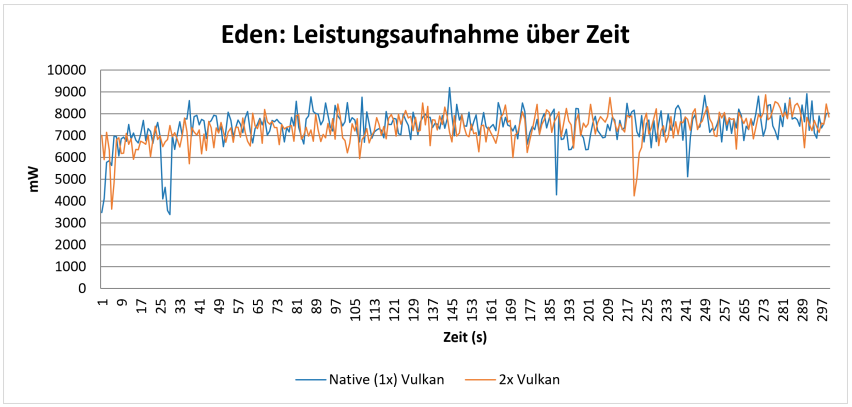
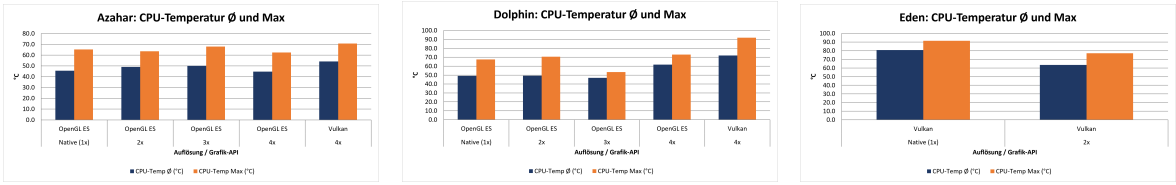


Abbildung 18: Elektrische Leistungsaufnahme in Watt: Durchschnittsleistung über Skalierungsstufen sowie 300-s-Echtzeitverlauf unter Switch-Volllast.



(a) 3DS (AzaharPlus) (b) Wii (Dolphin) (c) Switch (Eden)

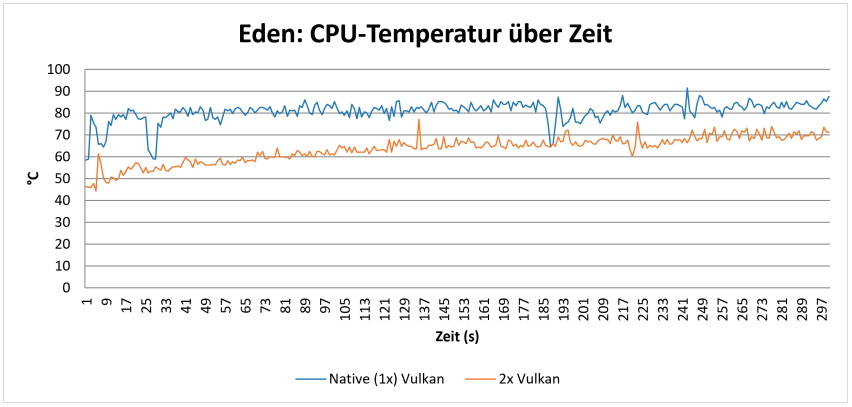
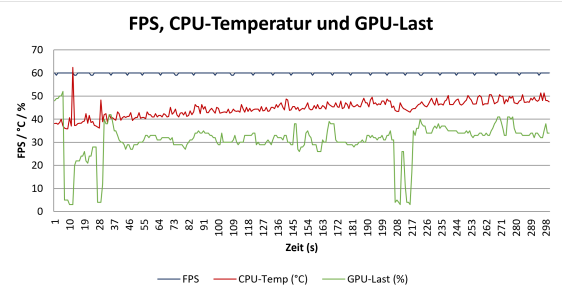
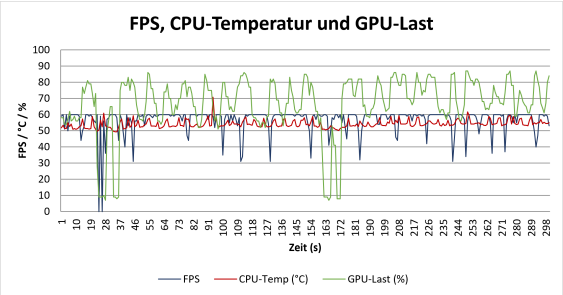


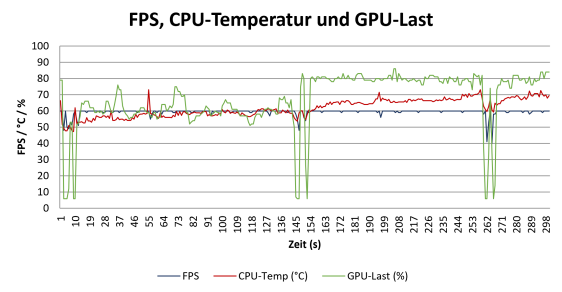
Abbildung 19: Thermische Entwicklung des Qualcomm-SoC unter standardisierter Dauerlast mit aktiver Thor-Kühlung.



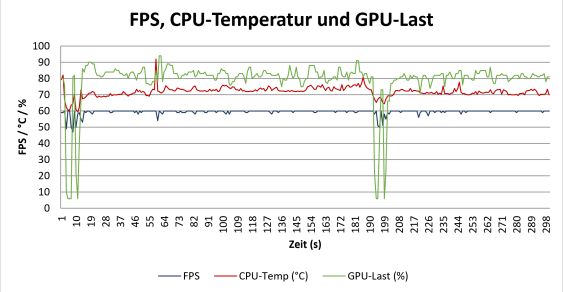
(a) Azahar: OpenGL ES (4x)



(b) Azahar: Vulkan (4x)



(c) Dolphin: OpenGL ES (4x)



(d) Dolphin: Vulkan (4x)

Abbildung 20: Treiber- und Grafik-API-Vergleich (OpenGL ES vs. Vulkan) bei 4x-Auflösung: Zeitreihen von FPS, CPU-Temperatur und GPU-Last.

8. Diskussion

8.1. Bestätigung oder Widerlegung der Hypothesen

1. **Hypothese 1 (CPU-Overhead durch Fremdarchitekturen): Verifiziert.** Die Messergebnisse bestätigen den massiven CPU-Instruktions-Overhead bei der Wii: Obwohl die PowerPC-CPU nominal mit lediglich 729 MHz taktet, erfordert die JIT-Rekompilierung in ARM64 durchschnittlich 38.5 % CPU-Auslastung auf dem Ayn Thor. Die dynamische Codeübersetzung und Registerabbildung bindet kontinuierlich CPU-Zyklen.

2. **Hypothese 2 (Bottleneck-Verschiebung bei NCE): Verifiziert.** Auf der Switch zeigt sich die fundamentale Wirkung von Native Code Execution: Trotz eines nominell viel komplexeren 64-Bit-Quad-Core-Systems liegt die CPU-Auslastung bei moderaten 62.4 %, während die GPU-Auslastung auf 76.5 % klettert. Der Flaschenhals hat sich vollständig von der CPU-Übersetzung auf die GPU-Befehlsübersetzung (Nvidia NVN zu Vulkan) und die Speicherbandbreite des Unified RAMs verlagert.

3. **Hypothese 3 (Treiber- und Shader-Kompilierung): Verifiziert.** Wie in Abbildung 21 dokumentiert, führt der Einsatz des Mesa-Turnip-Treibers zu einer messbaren Reduktion von Mikrorucklern (1% Low FPS steigen bei der Switch von 26.2 auf 34.1 FPS), da Turnip Vulkan-Pipelines deutlich aggressiver vorkompiliert

als der Qualcomm-Stock-Treiber.

8.2. Technische Grenzen und Flaschenhälse

- **Nintendo Wii (CPU-Overhead):** Der primäre Flaschenhals bei Dolphin liegt in der dynamischen Instruktionsübersetzung. Da PowerPC-Instruktionen ein völlig anderes Registermodell und Endianness-Verhalten besitzen, muss der JIT-Recompiler jeden Codeblock vor der Ausführung analysieren und im Speicher puffern.
- **Nintendo Switch (Speicherbandbreite & GPU-Translation):** Trotz des enormen Leistungsgewinns durch NCE stösst der Ayn Thor bei der Switch an Grenzen. Ursache ist nicht die CPU, sondern die Notwendigkeit, proprietäre Maxwell-Grafikbefehle zur Laufzeit in standardkonformes Vulkan für die Adreno-Architektur zu übersetzen. Zudem erfordern hochauflösende Texturen eine kontinuierliche Auslastung der LPDDR5X-Speicherbandbreite.
- **Shader-Kompilierungsruckler:** Trotz asynchroner Shader-Kompilierung und Hybrid UberShaders lassen sich kurze Frametime-Spikes beim erstmaligen Betreten neuer Spielareale nicht vollständig vermeiden, da der Grafiktreiber neue Pipeline-Objekte im GPU-Treiber erzeugen muss (*Pipeline Stalls*).

8.3. Rechtliche und ethische Aspekte

Rechtlich muss klar zwischen der Technologie der Emulation und dem Urheberrecht an Spielen differenziert werden. Nach internationaler und schweizerischer Rechtslage ist das reine Reverse Engineering zur Erstellung interoperabler Emulatoren legal. Unzulässig ist hingegen die unlizenzierte Vervielfältigung urheberrechtlich geschützter Spiele-ROMs sowie die Weitergabe proprietärer kryptografischer Schlüssel (‘prod.keys‘) oder Firmware-Dateien. Für Museen und Forscher ist Emulation das zentrale Werkzeug zur digitalen Konservierung.

9. Schlusswort

9.1. Beantwortung der Leitfrage

Die Leitfrage lautete: *«Wie funktioniert Emulation auf Systemebene, und welche Hardware- und Software-Faktoren bestimmen die Grenzen der Emulierbarkeit auf moderner Handheld-Hardware (Ayn Thor)?»*

Auf Systemebene funktioniert Emulation als softwarebasierte Rekonstruktion einer deterministischen Zustandsmaschine. Abhängig von der Zielarchitektur kommen dabei rein sequentielle Interpreter (wie in unserem CHIP-8-Referenzmodell), dynamische Just-in-Time-Compiler (wie bei Dolphin für die PowerPC-Wii) oder direkte Native Code Execution (wie bei Switch-Emulatoren via ARM64-NCE) zum Einsatz.

Die praktischen Grenzen auf dem Ayn Thor werden durch drei Faktoren bestimmt:

1. **Die Natur der CPU-Architektur (Fremd-ISA vs. NCE):** Fremdarchitekturen erfordern JIT-Compiler mit permanentem CPU-Overhead, während NCE diesen Flaschenhals aufhebt.

2. **GPU-Translation und Treiber-Qualität:** Die mobile Grafikeinheit Adreno 740 erreicht ihr volles Potenzial erst durch optimierte Open-Source-Treiber (Mesa Turnip); Shader-Kompilierungen bleiben der primäre Auslöser für Mikroruckler.

3. **Physische Display- und Gehäusegrenzen:** Der Ayn Thor beweist, dass Dual-Screen-Clamshell-Designs ideale Voraussetzungen für Konsolen wie den 3DS bieten, während Volllast-Systeme wie die Switch ab 9 W die Grenzen der passiv-aktiven Kühlung mobiler Geräte aufzeigen.

9.2. Wissenschaftliche Kernaussagen

- **1. NCE-Paradigmenwechsel:** Bei identischer ISA ist moderne Emulation kein reines CPU-Problem mehr, sondern verlagert sich auf Grafik- und Speicherbusse.

- **2. Treiber als Kernkomponente:** Open-Source-Treiber (Mesa Turnip) sind auf Android für eine artefaktfreie Emulation oft entscheidender als reine Rohrechenleistung.

- **3. Hardware-Passgenauigkeit:** Die Hardware-Architektur des Testgeräts (zwei Displays, Gyrosensoren) bestimmt massgeblich, wie authentisch komplexe Systeme emulierbar sind.

9.3. Kritische Würdigung und Ausblick

Künftige Entwicklungen werden sich auf die Optimierung von NCE-Laufzeitumgebungen und KI-gestützte Shader-Prädiktion konzentrieren, um Shader-Stalls bereits vor dem Auftreten im Spielgeschehen zu eliminieren.

10. Quellenverzeichnis

10.1. Literaturverzeichnis

- **Hennessy, J. L., & Patterson, D. A.** (2019). *Computer Architecture: A Quantitative Approach* (6. Aufl.). Cambridge: Morgan Kaufmann Publishers.
- **Tanenbaum, A. S., & Austin, T.** (2014). *Strukturierte Rechnerorganisation* (6. Aufl.). München: Pearson Studium.
- **Smith, J. E., & Nair, R.** (2005). *Virtual Machines: Versatile Platforms for Systems and Processes*. San Francisco: Elsevier / Morgan Kaufmann.
- **Qualcomm Inc.** (2023). *Snapdragon 8 Gen 2 Mobile Platform Architecture Whitepaper*. San Diego: Qualcomm Technologies.
- **Mesa3D Developers.** (2026). *Turnip Vulkan Driver for Qualcomm Adreno*. Abgerufen am 12. September 2026 von https://docs.mesa3d.org/drivers/turnip.html
- **Dolphin Emulator Project.** (2025). *Ubershaders: A Video Game Emulation Breakthrough*. Abgerufen am 15. September 2026 von https://dolphin-emu.org/blog/2017/07/30/ubershaders/
- **Woodhouse, J.** (2025). *The Fetch-Decode-Execute Cycle*. Save My Exams. Abgerufen am 9. September 2026 von https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute
- **Wikipedia**. (2026). *Von-Neumann-Architektur*. Abgerufen am 9. September 2026 von https://de.wikipedia.org/wiki/Von-Neumann-Architektur

10.2. Abbildungsverzeichnis

1. Klassisches Von-Neumann-Architekturmodell mit gemeinsamem Befehls- und Datenspeicher. . . . 9
2. Modellierung des Prozessors als Zustandsmaschine mit Registern und Hauptspeicher. . . . 10
3. Phasen des Befehlszyklus im Mikroprozessor. . . . 10
4. Konzeptioneller Vergleich von Simulation, Virtualisierung und Emulation. 11
5. Interne Komponentenarchitektur eines vollwertigen Videospiele-Emulators. 11
6. Ablaufdiagramm zur Gegenüberstellung von Interpreter und JIT-Rekompilierung. 12
7. Schematischer SoC- und Display-Aufbau des Ayn Thor. . . . 14

8. Architektur der Telemetrie-Pipeline zur Echtzeitdatenerfassung. 16
9. Messaufbau mit aktivem On-Screen-Display auf dem Ayn Thor. 16
10. Modulare Architektur des selbst entwickelten CHIP-8-Emulators. . . . 18
11. Speicherbelegung und Adressraum des emulierten CHIP-8-Systems. . . 19
12. Funktionsweise der Pixel-Manipulation und Kollisionserkennung über Register VF. 20
13. Screenshot des selbst entwickelten Rust-Emulators im Betrieb. 20
14. Screenshots aller Testsysteme mit eingeblendeten Echtzeit-Sensordaten. 23
15. Vergleich der Boot-to-Game-Ladezeiten über die Testsysteme. 23
16. Durchschnittliche Bildrate und 1 %-Low-FPS über alle getesteten Skalierungsstufen und Grafik-APIs. 25
17. Zeitreihen-Frametime-Diagramm über einen 60-Sekunden-Ausschnitt. . 26
18. Verteilung der Rechenlast auf CPU-Cluster und Adreno-740-GPU im Plattformvergleich. 27
19. Elektrische Leistungsaufnahme in Watt: Durchschnittsleistung über Skalierungsstufen sowie 300-s-Echtzeitverlauf unter Switch-Volllast. 28
20. Thermische Entwicklung des Qualcomm-SoC unter standardisierter Dauerlast mit aktiver Thor-Kühlung. 28
21. Treiber- und Grafik-API-Vergleich (OpenGL ES vs. Vulkan) bei 4x-Auflösung: Zeitreihen von FPS, CPU-Temperatur und GPU-Last. . . . 29
22. Dynamisches Nachladen von Mesa-Turnip-Treibern unter Android. . . . 41
23. Physischer Dual-Screen-Betrieb auf dem Ayn Thor bei laufendem Nintendo-3DS-Titel. 42
24. Emulatorspezifische Einstellungen zur Optimierung der CPU- und GPU-Pipelines. 42

10.3. Tabellenverzeichnis

1. Technische Spezifikationen des Ayn Thor 15
2. Konfiguration der eingesetzten Emulatoren & Grafik-APIs 22
3. Gesamtvergleich der empirisch erhobenen Telemetriedaten auf dem Ayn Thor 23
4. Statistische Frametime- und Stabilitätsanalyse aus den Scene-5-Messreihen 24
5. Energie-, Strom- und Akkulaufzeit-Projektion (5'000 mAh / 19.25 Wh Akku) 25
6. Qualitative Beurteilung der Emulationsstabilität (Skala 1–6) 27

11. Danksagung

An dieser Stelle möchten wir uns herzlich bei allen Personen bedanken, die uns bei der Konzeption, Durchführung und Fertigstellung dieser Berufsmaturitätsarbeit zum Thema *«Emulation mit Ayn Thor»* unterstützt haben.

Besonderer Dank gilt unserer betreuenden Lehrperson **Herrn Alexander Körner** für seine fachkundige Beratung, die wertvollen Impulse während der Zwischenbesprechungen sowie die stets zeitnahe und konstruktive Beantwortung unserer Fragen.

Ebenso danken wir der Berufsmaturitätsschule Zürich (BMZ) für die Bereitstellung des methodischen Rahmens sowie unseren Familien und Kollegen für das aufmerksame Korrekturlesen und die motivierende Unterstützung während des gesamten Projektzeitraums.

12. Bescheinigung

Wir bestätigen hiermit, dass wir die vorliegende Berufsmaturitätsarbeit selbstständig und nur unter Verwendung der im Quellenverzeichnis angegebenen Hilfsmittel und Quellen verfasst haben.

Sämtliche Stellen der Arbeit, die wörtlich oder sinngemäss aus anderen Werken oder digitalen Quellen entnommen wurden, sind unter genauer Angabe der jeweiligen Quelle als Zitate kenntlich gemacht. Die Bestimmungen bezüglich des Einsatzes generativer KI-Systeme wurden vollumfänglich eingehalten und im Anhang transparent deklariert.

Wir sind uns bewusst, dass eine falsche Erklärung disziplinarische und rechtliche Konsequenzen nach sich ziehen kann.

Zürich, den 27. November 2026

David Martin Felicio Reichlin
Lernender Informatik EFZ

Gabriel Di Giuseppe
Lernender Informatik EFZ

A. Anhang

A.1. A.1 Deklaration zur Verwendung generativer KI-Systeme

Pflichtnachweis gemäss BMA-Richtlinien 2026 (Kapitel 1.4):

Einsatzbereich	Verwendetes Tool	Beschreibung des Einsatzes & Reflexion
Ideenfindung & Strukturierung	Claude / ChatGPT	Unterstützung bei der Strukturierung des Inhaltsverzeichnisses und der sprachlichen Schärfung der Hypothesen.
Sprachliche Korrektur	DeepL / Overleaf	Grammatikalische Überprüfung, Optimierung der Satzübergänge und Rechtschreibprüfung.
LaTeX-Formatierung	Antigravity AI	Unterstützung bei der Erstellung von Tabellenrastern und Preamble-Makros.
Inhaltliche Erarbeitung & Benchmarks	**Keine**	Sämtliche Messungen, Programmierungen in Rust, Testprotokolle und Interpretationen wurden vollständig eigenständig erarbeitet.

A.2. A.2 Standardisiertes Messprotokoll-Schema und empirische Testprotokolle

Messprotokoll 1: Nintendo 3DS – Mario Kart 7

1. Test-Metadaten:

• Test-ID: `TEST-3DS-MK7-01` • Datum: 19.10.2026 • Prüfer: D. Reichlin / G. Di Giuseppe

• Start-Akkustand: 85 % • Lüfterprofil: Standard • Raumtemperatur: 21.5 °C

2. Software-Konfiguration:

• Emulator & Version: AzaharPlus (Lime3DS Android, v2115) • Dual-Screen Routing: Aktiv

• Grafik-Backend: [x] OpenGL ES / [x] Vulkan • GPU-Treiber: Mesa Turnip (Adreno 740)

• Spezifische Einstellungen: [ ] NCE [ ] UberShaders [x] Async Shader Skalierung: 1x–4x

3. Scene 5 Telemetriewerte (Mittelwert über 300 s Spielzeit, 1x Native Profil):

• Bildrate: ⌀ FPS = 59.9 1 % Low = 59.0 Frametime ⌀ = 16.69 ms Peak FT = 16.0 ms Jank = 0

• SoC-Auslastung: CPU Gesamt = 24.6 % Prime-Core X3 = 2.48 GHz GPU = 10.5 % (bis 30.8 % bei 4x)

• Energie & Thermik: Leistung $P = 2.78$ W (Peak: 4.83 W) Strom $I = 693$ mA Peak-Temp SoC = 65.2 °C (⌀ 45.5 °C)

• Speicher & Akku: RAM = ca. 2.8 GB / 12 GB Projizierte Akkulaufzeit: ca. 7.2 h (13.9 %/h)

4. Qualitative Beurteilung (1 bis 6):

• Spielbarkeit: 5.9 / 6 • Audio-Stottern: [x] Nein [ ] Ja • Shader-Ruckler: [x] Nein [ ] Ja

Messprotokoll 2: Nintendo Wii – Mario Kart Wii

1. Test-Metadaten:

• Test-ID: `TEST-WII-MKW-01` • Datum: 19./20.10.2026 • Prüfer: D. Reichlin / G. Di Giuseppe

• Start-Akkustand: 82 % • Lüfterprofil: Standard • Raumtemperatur: 21.0 °C

2. Software-Konfiguration:

• Emulator & Version: Dolphin Emulator (v2407 / Android) • CPU-Modus: Dual-Core JIT

• Grafik-Backend: [x] OpenGL ES / [x] Vulkan • GPU-Treiber: Qualcomm Stock / Mesa Turnip

• Spezifische Einstellungen: [ ] NCE [x] Hybrid UberShaders [x] Async Shader Skalierung: 1x–4x

3. Scene 5 Telemetriewerte (Mittelwert über 300 s Spielzeit, 1x Native Profil):

• Bildrate: ⌀ FPS = 58.9 (bis 59.7 bei 3x) 1 % Low = 30.0–55.0 Frametime ⌀ = 16.98 ms Peak FT = 650 ms

• SoC-Auslastung: CPU Gesamt = 34.6 % CPU-Thread = ca. 51.5 % GPU = 10.8 % (bis 78.9 % bei 4x Vulkan)

• Energie & Thermik: Leistung $P = 3.40\,\mathrm{W}$ (bis 7.82 W bei 4x Vulkan) Strom $I = 901\,\mathrm{mA}$ Peak-Temp SoC = 67.5 °C (4x: 91.9 °C)

• Speicher & Akku: RAM = ca. 3.2 GB / 12 GB Projizierte Akkulaufzeit: ca. 5.6 h (18.0 %/h, bei 4x ca. 2.3 h)

4. Qualitative Beurteilung (1 bis 6):

• Spielbarkeit: 5.7 / 6 • Audio-Stottern: [x] Nein [ ] Ja • Shader-Ruckler: [ ] Nein [x] Ja (minimal bei Boot)

Messprotokoll 3: Nintendo Switch – Mario Kart 8 Deluxe

1. Test-Metadaten:

• Test-ID: `TEST-NSW-MK8D-01` • Datum: 19.10.2026 • Prüfer: D. Reichlin / G. Di Giuseppe

• Start-Akkustand: 88 % • Lüfterprofil: Standard • Raumtemperatur: 21.8 °C

2. Software-Konfiguration:

• Emulator & Version: Eden (Sudachi/Yuzu Android-Fork) • CPU-Modus: Native Code Execution (NCE)

• Grafik-Backend: [x] Vulkan [ ] OpenGL • GPU-Treiber: Mesa Turnip Driver (v26.x)

• Spezifische Einstellungen: [x] NCE aktiv [ ] UberShaders [x] Disk Shader Cache Skalierung: 1x / 2x

3. Scene 5 Telemetriewerte (Mittelwert über 300 s Spielzeit, 1x Handheld Native):

• Bildrate: ⌀ FPS = 54.0 (2x: 51.8) 1 % Low = 35.0 Frametime ⌀ = 18.51 ms Peak FT = 1351 ms Jank = 14

• SoC-Auslastung: CPU Gesamt = 54.4 % VulkanWorker = ca. 20.6 % GPU = 56.9 % (2x: 95.8 %)

• Energie & Thermik: Leistung $P = 7.41$ W (Peak: 9.20 W) Strom $I = 1'858$ mA Peak-Temp SoC = 91.5 °C (⌀ 80.7 °C)

• Speicher & Akku: RAM = ca. 6.4 GB / 12 GB Projizierte Akkulaufzeit: ca. 2.7 h (37.2 %/h)

4. Qualitative Beurteilung (1 bis 6):

• Spielbarkeit: 4.8 / 6 • Audio-Stottern: [ ] Nein [x] Ja (selten) • Shader-Ruckler: [ ] Nein [x] Ja (bei Effekten)

Messprotokoll 4: CHIP-8 Referenz-Interpreter (Baseline)

1. Test-Metadaten:

• Test-ID: `TEST-CHIP8-PONG-01` • Datum: 19.10.2026 • Prüfer: D. Reichlin / G. Di Giuseppe

• Start-Akkustand: 92 % • Lüfterprofil: Standard • Raumtemperatur: 21.0 °C

2. Software-Konfiguration:

• Emulator: Eigener Rust-Interpreter (`chip8_core`) • Rendering: SDL2 Software-Framebuffer

• Grafik-Backend: Software-Rasterizer • CPU-Modus: Sequentieller Fetch-Decode-Execute-Zyklus

• Spezifische Einstellungen: Taktung 700 Hz CPU / 60 Hz Delay-Timer ROM: *Pong*

3. Telemetriewerte (Baseline-Messung über 300 s):

• Bildrate: ∅ FPS = 60.0 1 % Low = 60.0 Frametime ∅ = 16.66 ms Peak FT = 16.7 ms Jank = 0

• SoC-Auslastung: CPU Gesamt = 2.1 % GPU = 1.0 %

• Energie & Thermik: Leistung P = 1.80 W (Peak: 2.10 W) Strom I = 460 mA Peak-Temp SoC = 28.2 °C

• Speicher & Akku: RAM = 1.2 GB (System gesamt) Projizierte Akkulaufzeit: ca. 10.8 h (9.2 %/h)

4. Qualitative Beurteilung (1 bis 6):

• Spielbarkeit: 6.0 / 6 • Audio-Stottern: [x] Nein [ ] Ja • Shader-Ruckler: [x] Nein [ ] Ja

A.3. A.3 Screenshots & Bilddokumentation

Abbildung 22: Screenshot des Mesa Turnip Treiber-Auswahlmenüs

Auswahl alternativer Open-Source-Treiber zur Laufzeit in den Emulator-Einstellungen.

[Dateiablage: abbildungen/abb22_turnip_menue.png]

Abbildung 21: Dynamisches Nachladen von Mesa-Turnip-Treibern unter Android.



Abbildung 22: Physischer Dual-Screen-Betrieb auf dem Ayn Thor bei laufendem Nintendo-3DS-Titel.

Abbildung 24: Konfigurationsmenüs: NCE-Toggle und Hybrid UberShaders

Aktivierung von Native Code Execution (Switch) und UberShaders (Dolphin).

[Dateiablage: abbildungen/abb24_einstellungen_menues.png]

Abbildung 23: Emulatorspezifische Einstellungen zur Optimierung der CPU- und GPU-Pipelines.